研究生课程报告

课程名称	微处理器架构
题目名称	五级流水线 **RISC-V** 处理器的设计与验证
班级	
组长	
组员	

摘要

CPU 是计算机系统的核心部件，在各类信息终端中得到了广泛的应用。处理器的设计及制造技术也是计算机技术的核心之一。本次 CPU 设计实现了 RISC-V 的单周期处理器设计和五级流水线处理器设计，该报告内容主要分为三部分：

（1）指令集架构设计。从寄存器定义、立即数扩展模式、寻址模式、寻址空间和指令集编码对 RISC-V 指令集进行了系统描述。本次指令集设计所用的存储方式为 load-store 类型，支持字节对齐方式，采用小端模式存储数据，支持常用的三种寻址方式，支持的操作数类型为整型，定点数操作，支持常用的算术运算和逻辑运算，且所有指令均采用 32 位定长编码。

（2）处理器设计。中央处理器（CPU）是计算机取指令和执行指令的部件，它主要由算术逻辑单元（ALU）、寄存器和控制器组成。对于单周期处理器设计，有指令存储器、数据存储器、寄存器堆、译码模块、ALU 模块、PC 寄存器等模块。对于五级流水线处理器的设计有 PC 寄存器、Control 模块、ALU 模块、ALU Control 模块、指令存储器、寄存器堆、数据寄存器、前递模块、冒险检测模块等。

（3）测试验证。对于单条指令的执行情况，我们通过多次汇编代码编写对单条指令进行下板验证；而对于 CPU 功能的总体测试，我们通过递归求累加和的方式对其进行验证，且下板验证在数码管上显示最终结果。

关键词： 指令集；CPU；RISC-V；流水线

ABSTRACT

CPU is the core component of computer systems and is widely used in various types of information terminals. Processor design and manufacturing technology is also one of the core of computer technology. This CPU design implements the single-cycle processor design of RISC-V and the five-stage pipeline processor design. The report is divided into three main parts.

（1）Instruction set architecture design. The RISC-V instruction set is systematically described in terms of register definition, immediate expansion mode, addressing mode, addressing space and instruction set encoding. The instruction set design uses the load-store type of storage, supports byte alignment, uses small-end mode to store data, supports the three commonly used addressing modes, supports integer and fixed-point operations, supports common arithmetic and logical operations, and all instructions are encoded with 32-bit fixed-length codes.

（2）Processor design. The central processing unit (CPU) is the part of the computer that takes instructions and executes them, and it consists mainly of arithmetic logic units (ALUs), registers, and controllers. For single-cycle processor design, there are instruction memory, data memory, register heap, decoding module, ALU module, PC registers and other modules. For a five-stage pipeline processor design there are PC registers, Control module, ALU module, ALU Control module, instruction memory, register heap, data registers, forwarding module, and adventure detection module.

（3）Test verification. For the execution of single instructions, we tested 18 instructions such as addi, slli, add, sw, jal, etc. and all of them can be executed normally; while for the overall test of CPU function, we verified it by recursive summation, and the final result is displayed on the digital tube under the board verification.

Keywords: Instruction set; CPU; RISC-V; Pipeline

目录

1. RISC-V 指令集架构设计（ISA） 1
1.1 指令集设计 1
1.2 寄存器定义 2
1.3 立即数扩展模式 3
1.4 寻址模式 3
1.5 寻址空间 3
1.6 字节对齐 4
1.7 指令集编码 4
1.7.1 移位运算 4
1.7.2 算术运算 5
1.7.3 逻辑运算 6
1.7.4 比较 7
1.7.5 分支 7
1.7.6 跳转和链接 8
1.7.7 加载 9
1.7.8 存储 9
2. RISC-V 处理器设计 12
2.1 单周期处理器顶层设计 12
2.1.1 指令存储器 14
2.1.2 数据存储器 14
2.1.3 寄存器堆 14
2.1.4 译码模块 14
2.1.5 ALU 模块 14
2.1.6 PC 寄存器 15
2.2 单周期处理控制器信号产生 15
2.3 信号在五级流水线中的传递 15

2.4 五级流水线 RISC-V 处理器设计 16
2.4.1 顶层模块名 17
2.4.2 顶层 I/O 定义 17
2.4.3 顶层结构图 18
3. 子模块设计 20
3.1 PC 寄存器设计 20
3.2 Control 设计 20
3.3 ALU 设计 21
3.4 ALU control 设计 22
3.5 指令存储器设计 23
3.6 寄存器堆设计 23
3.7 数据存储器设计 24
3.8 前递单元设计 24
3.9 冒险检测设计 26
4 RISC-V 处理器仿真验证 29
4.1 验证方案描述 29
4.2 测试用例 29
4.2.1 单条指令测试 29
4.2.2 应用程序测试 37
5. 项目管理 40
5.1 开发环境 40
5.2 时间安排 40
5.3 人员分工 40
5.4 过程记录 41
6. 课程总结 41
参考文献 42

1. RISC-V 指令集架构设计（ISA）

1.1 指令集设计

R 型指令：用于寄存器-寄存器操作。

I 型指令：用于立即数和访存 load 操作。

S 型指令：用于访存 store 操作。

B 型指令：用于条件跳转操作。

U 型指令：用于长立即数。

J 型指令：用于无条件跳转。

指令均为 32 位长，risc-v 提供 32 个寄存器操作数。所有指令要读写的寄存器标识符总是在同一位置（解码之前就可以开始访问寄存器）。

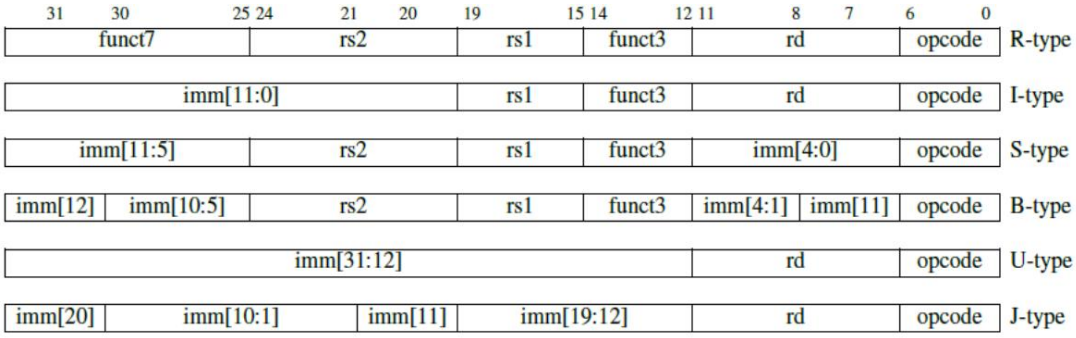

图 1.1 六种指令编码格式

1.2 寄存器定义

x0/zero	Hardwired zero	常数0
x1/ra	Return address	返回地址
x2/sp	Static pointer	栈指针
x3/gp	Global pointer	全局指针
x4/tp	Thread pointer	线程指针
x5/t0	Temporary	临时寄存器
x6/t1	Temporary	临时寄存器
x7/t2	Temporary	临时寄存器
x8/s0/fp	Saved regester,frame pointer	保存寄存器
x9/s1	Saved regester	保存寄存器
x10/a0	Function argument,return value	函数参数和结果
x11/a1	Function argument,return value	函数参数和结果
x12/a2	Function argument	函数参数和结果
x13/a3	Function argument	函数参数和结果
x14/a4	Function argument	函数参数和结果
x15/a5	Function argument	函数参数和结果
x16/a6	Function argument	函数参数和结果
x17/a7	Function argument	函数参数和结果
x18/s2	Saved regester	保存寄存器
x19/s3	Saved regester	保存寄存器
x20/s4	Saved regester	保存寄存器
x21/s5	Saved regester	保存寄存器
x22/s6	Saved regester	保存寄存器
x23/s7	Saved regester	保存寄存器
x24/s8	Saved regester	保存寄存器
x25/s9	Saved regester	保存寄存器
x26/s10	Saved regester	保存寄存器
x27/s11	Saved regester	保存寄存器
x28/t3	Temporary	临时寄存器
x29/t4	Temporary	临时寄存器
x30/t5	Temporary	临时寄存器
x31/t6	Temporary	临时寄存器
pc		程序计数器

图 1.2 寄存器

其中，X0 为常数 0，X1 为返回地址，X2 为栈指针，X3 为全局指针，x4 为线程指针，X5 到 X7 为临时寄存器，X8-9 为保存寄存器，X10 到 X17 为函数参数和结果，X18 到 X27 为保存寄存器，X28 到 X31 为临时寄存器，PC 为程序计数器。

1.3 立即数扩展模式

功能：对立即数进行扩展，根据指令源码并自动识别立即数并将其扩展为32位。

信号说明：

表一：立即数扩展设计信号说明

参数名	方向	位宽	说明
Instruction	输入	32	指令源码
imm	输出	32	32位立即数

详细操作说明：将指令源码的立即数进行提取，并扩展为32位的立即数。

1.4 寻址模式

A. 寄存器寻址

eg：SLLI:rd,rs1,shamt——立即数逻辑左移，把寄存器x[rs1]左移shamt位存入寄存器x[rd]中。空位补0，在RV32I中，shamt[5]=0时，指令为有效。

B.立即数寻址

eg：ORI:rd,rs1,imm——立即数取或，把寄存器x[rs1]和有符号位扩展的立即数imm按位取或，结果写入x[rd]。

C.位移量寻址

eg：ADDI:rd,rs1,imm——加立即数，把符号位扩展的立即数加到寄存器x[rs1]上，结果写入x[rd]中，忽略算数溢出。

1.5 寻址空间

寻址空间一般指的是CPU对于内存寻址的能力。通俗地说，就是能最多用到多少内存的一个问题，即能够寻址的最大容量。一般用MB和G来表示。

CPU 最大能查找多大范围的地址叫做寻址能力，CPU 的寻址能力以字节为单位，如 32 位寻址的 CPU 可以寻址 2 的 32 次方大小的地址也就是 4G，这也是为什么 32 位的 CPU 最大能搭配 4G 内存的原因，再多的话 CPU 就找不到了。

一般情况下，内存容量越大，处理数据的能力也就越强，但内存容量不可能无限的大，它要受到系统结构、硬件设计、制造成本等多方面因素的制约，一个最直接的因素取决于系统的地址总线的地址寄存器的宽度（位数）。 计算机的寻找范围由总线宽度（处理器的地址总线的位数）决定的，也可以理解为 cpu 寄存器位数，这二者一般是匹配的。

1.6 字节对齐

字节对齐的作用不仅是便于 cpu 快速访问，同时合理的利用字节对齐可以有效地节省存储空间。

对于 32 位机来说，4 字节对齐能够使 cpu 访问速度提高，比如说一个 long 类型的变量，如果跨越了 4 字节边界存储，那么 cpu 要读取两次，这样效率就低了。但是在 32 位机中使用 1 字节或者 2 字节对齐，反而会使变量访问速度降低。所以这要考虑处理器类型，另外还得考虑编译器的类型。

1.7 指令集编码

1.7.1 移位运算

表二：移位运算操作

Shift Left Logical	R	SLL	rd,rsl,rs2
Shift Left Log.Imm.	I	SLLI	rd,rsl,shamt
Shift Right Logical	R	SRL	rd,rsl,rs2
Shift RightLog.Imm	I	SRLI	rd,rsl,shamt
Shift Right Arithmetic	R	SRA	rd,rsl,shamt
Shift Right Arith.Imm	I	SRAI	rd,rsl,shamt

SLL:rd,rs1,rs2——逻辑左移，把寄存器 x[rs1]左移 x[rs2]位，空出的位置填入 0，结果写入 x[rd]。x[rs2]的低 5 位（如果是 RV64I 则是低 6 位）代表移动位数，其高位则被忽略。 x[rd] = x[rs1] ≪ x[rs2]　寻址方式：寄存器寻址

SLLI:rd,rs1,shamt——立即数逻辑左移，把寄存器 x[rs1]左移 shamt 位存入寄存器 x[rd]中。空位补 0，在 RV32I 中，shamt[5]=0 时，指令为有效。 x[rd] = x[rs1] ≪ shamt 寻址方式：寄存器寻址、立即数寻址　　立即数大小：0~4095

SRL:rd,rs1,rs2——逻辑右移，把寄存器 x[rs1]右移 x[rs2]位，空出的位置填入 0，结果写入 x[rd]。x[rs2]的低 5 位（如果是 RV64I 则是低 6 位）代表移动位数，其高位则被忽略。x[rd] = (x[rs1] ≫ x[rs2])　　寻址方式：寄存器寻址

SRLI:rd,rs1,shamt——立即数逻辑右移，把寄存器 x[rs1]右移 shamt 位存入寄存器 x[rd]中。x[rd] = (x[rs1] ≫ shamt)　　寻址方式：寄存器寻址、立即数寻址

立即数大小：0~4095

SRA:rd,rs1,rs2——算数右移，把寄存器 x[rs1]右移 x[rs2]位，空位用 x[rs1]的最高位填充，结果写入 x[rd]。x[rs2]的低 5 位为移动位数，高位则被忽略。x[rd] = (x[rs1] ≫ x[rs2])　　寻址方式：寄存器寻址

SRAI:rd,rs1,shamt——立即数算数右移，把寄存器 x[rs1]右移 shamt 位，空位用 x[rs1]的最高位填充，结果写入 x[rd]。x[rd] = (x[rs1] ≫*s* shamt)　　寻址方式：寄存器寻址、立即数寻址。立即数大小：0~4095

1.7.2 算术运算

表三：算术运算操作

Arithmetic ADD	R	ADD	rd,rsl,rs2
ADD Immediate	I	ADDI	rd,rsl,imm
SUBtract	R	SUB	rd,rsl,rs2

ADD:rd,rs1,rs2——加，把寄存器 x[rs2]加到寄存器 x[rs1]上，写入寄存器 x[rd]中，忽略算数溢出。x[rd] = x[rs1] + x[rs2]　寻址方式：寄存器寻址

ADDI:rd,rs1,imm——加立即数，把符号位扩展的立即数加到寄存器 x[rs1]

上，结果写入 x[rd]中，忽略算数溢出。 x[rd] = x[rs1] + sext(immediate) 寻址方式：寄存器寻址、立即数寻址 立即数大小：0~4095

SUB:rd,rs1, rs2——减，x[rs1]减去 x[rs2]，结果写入 x[rd]。忽略算术溢出。寻址方式：寄存器寻址

1.7.3 逻辑运算

表四：逻辑运算操作

Logical XOR	R	XOR	rd,rsl,rs2
XOR	I	XORI	rd,rsl,imm
OR	R	OR	rd,rsl,rs2
OR Immediate	I	ORI	rd,rsl,imm
AND	R	AND	rd,rsl,rs2
AND Immediate	I	ANDI	rd,rsl,imm

XOR:rd,rs1,rs2——异或，x[rs1]和 x[rs2]按位异或，结果写入 x[rd]中。x[rd] = x[rs1] ^ x[rs2] 寻址方式：寄存器寻址

XORI:rd,rs1,imm——立即数异或 x[rs1]和有符号扩展的 imm 按位异或，结果写入 x[rd]。x[rd] = x[rs1] ^ sext(immediate) 寻址方式：寄存器寻址、立即数寻址，立即数大小：0~4095

OR:rd,rs1,rs2——取或，把寄存器 x[rs1]和寄存器 x[rs2]按位取或，结果写入 x[rd]。x[rd] = x[rs1] | x[rs2] 寻址方式：寄存器寻址

ORI:rd,rs1,imm——立即数取或，把寄存器 x[rs1]和有符号位扩展的立即数 imm 按位取或，结果写入 x[rd]。x[rd] = x[rs1] | sext(immediate) 寻址方式：寄存器寻址、立即数寻址 立即数大小：0~4095

AND:rd,rs1,rs2——与，将寄存器 x[rs1]和寄存器 x[rs2]按位与，结果写入 x[rd]。x[rd] = x[rs1] & x[rs2] 寻址方式：寄存器寻址

ANDI:rd,rs1,imm——与立即数，把符号位扩展的立即数加到寄存器 x[rs1]上，结果写入 x[rd]。忽略算术溢出。x[rd] = x[rs1] & sext(immediate) 寻址方式：寄存器寻址、立即数寻址 立即数大小：0~4095

1.7.4 比较

表五：比较操作

Compare Set<	R	SLT	rd,rsl,rs2
Set<Immediate	I	SLTI	rd,rsl,imm
Set<Unsigned	R	SLTU	rd,rsl,rs2
Set<Imm Unsigned	I	SLTIU	rd,rsl,imm

SLT:rd,rs1,rs2——小于则置位，比较 x[rs1]和 x[rs2]中的数，如果 x[rs1]更小，向 x[rd]写入 1，否则写入 0。 x[rd] = (x[rs1] <*s* x[rs2]) 寻址方式：寄存器寻址

SLTI:rd,rs1,imm——小于立即数则置位，比较 x[rs1]和有符号扩展的 imm，如果 x[rs1]更小，向 x[rd]写入 1，否则写入 0。 x[rd]= (x[rs1] <*s* sext(immediate))

寻址方式：寄存器寻址、立即数寻址 立即数大小：-2048~+2047

SLTU:rd,rs1,rs2——无符号小于则置位，比较 x[rs1]和 x[rs2]中的数，比较时视为无符号数，如果 x[rs1]更小，向 x[rd]写入 1，否则写入 0。x[rd] = (x[rs1] <*u* x[rs2])

寻址方式：寄存器寻址

SLTIU:rd,rs1,imm——无符号小于立即数则置位，比较 x[rs1]和有符号扩展的 imm，比较时视为无符号数，如果 x[rs1]更小，向 x[rd]写入 1，否则写入 0。

x[rd] = (x[rs1] <*u* sext(immediate))

寻址方式：寄存器寻址、立即数寻址 立即数大小：0~4095

1.7.5 分支

表六：分支操作

Branches Branch =	B	BEQ	rsl,rs2,imm
Branch ≠	B	BEN	rsl,rs2,imm
Branch <	B	BLT	rsl,rs2,imm
Branch ≥	B	BGE	rsl,rs2,imm
Branch < Unsigned	B	BLTU	rsl,rs2,imm
Branch ≥ Unsigned	B	BGEU	rsl,rs2,imm

寻址方式：寄存器寻址、立即数寻址

寻址空间：$\pm 2K$

立即数大小：-2048~+2047

BEQ:rs1,rs2,offset——相等时分支,若寄存器 x[rs1]和寄存器 x[rs2]的值相等，把 pc 的值设为当前值加上符号位扩展的偏移 offset。

if (rs1 == rs2) pc += sext(offset)

BNE:不等时分支,若寄存器 x[rs1]和寄存器 x[rs2]的值相等，把 pc 的值设为当前值加上符号位扩展的偏移 offset。

if (rs1 ≠ rs2) pc += sext(offset)

BLT:rs1,rs2,offset——小于时分支,若寄存器 x[rs1]的值小于寄存器 x[rs2]的值（均视为二进制补码），把 pc 的值设为当前值加上符号位扩展的偏移 offset。

if (rs1 <s rs2) pc += sext(offset)

BGE:rs1,rs2,offset——大于等于时分支,若寄存器 x[rs1]的值小于寄存器 x[rs2]的值（均视为二进制补码），把 pc 的值设为当前值加上符号位扩展的偏移 offset。

if (rs1 ≥s rs2) pc += sext(offset)

BLTU:rs1,rs2,offset——无符号小于时分支,若寄存器 x[rs1]的值小于寄存器 x[rs2]的值（均视为二进制补码），把 pc 的值设为当前值加上符号位扩展的偏移 offset。

if (rs1 <u rs2) pc += sext(offset)

BGEU:无符号大于等于时分支,若寄存器 x[rs1]的值小于寄存器 x[rs2]的值（均视为二进制补码），把 pc 的值设为当前值加上符号位扩展的偏移 offset。

if (rs1 ≥u rs2) pc += sext(offset)

1.7.6 跳转和链接

表七：跳转和链接操作

Jump & Link J&L	J	JAL rd,imm
Jump & Link Register	I	JALR rd,rsl,imm

JAL:rd,offset——跳转并链接,把下一条指令的地址(pc+4)给 rd，然后把 pc 设置为当前值加上符号位扩展的 offset。rd 默认为 x1。

寻址方式：寄存器寻址、立即数寻址

寻址空间：±512K

立即数大小：±512K

JALR:rd,offset(rs1)——跳转并寄存器链接,把 pc 设置为=x[rs1] + sign-extend(offset)，把计算出的地址的最低有效位设为 0，并将原 pc+4 的值写入 f[rd]。rd 默认为 x1。

寻址方式：寄存器寻址、立即数寻址

寻址空间：±512K

立即数大小：±512K

1.7.7 加载

表八：加载操作

Loads Load Word	I	LW rd,isl,imm

寻址方式：寄存器寻址、立即数寻址

寻址空间：$\pm 2K$

立即数大小：-2048 到+2047

LW:rd,offset(rs1)——字加载，从地址 x[rs1] + sign-extend(offset)读取四个字节，写入 x[rd]。对于 RV64I，结果要进行符号位扩展。

1.7.8 存储

表九：存储操作

Stores Store Word	S	SW rs1,rs2,imm

寻址方式：寄存器寻址、立即数寻址

寻址空间：$\pm 2K$

立即数大小：-2048~+2047

SW:rs2,offset(rs1)—存字，将 x[rs2]的低位个字节存入内存地址 x[rs1]+sign-extend(offset)。 M[x[rs1] + sext(offset) = x[rs2][31:0]

本次设计所用到的所有指令集及编码格式如表 10 所示：

表十：所用基本指令集

0000000	rs2	rs1	001	rd	0110011	SLL
0000000	shamt	rs1	001	rd	0010011	SLLI
0000000	rs2	rs1	101	rd	0110011	SRL
0000000	shamt	rs1	101	rd	0010011	SRLI
0100000	rs2	rs1	101	rd	0110011	SRA
0100000	shamt	rs1	101	rd	0010011	SRAI
0000000	rs2	rs1	000	rd	0110011	ADD
imm[11:0]		rs1	000	rd	0010011	ADDI
0100000	rs2	rs1	000	rd	0110011	SUB
0000000	rs2	rs1	100	rd	0110011	XOR
imm[11:0]		rs1	100	rd	0010011	XORI
0000000	rs2	rs1	110	rd	0110011	OR
imm[11:0]		rs1	110	rd	0010011	ORI
0000000	rs2	rs1	111	rd	0110011	AND
imm[11:0]		rs1	111	rd	0010011	ANDI
0000000	rs2	rs1	010	rd	0110011	SLT
imm[11:0]		rs1	010	rd	0010011	SLTI
0000000	rs2	rs1	011	rd	0110011	SLTU
imm[11:0]		rs1	011	rd	0010011	SLTIU
imm[12|10:5]	rs2	rs1	000	imm[4:1|11]	1100011	BEQ
imm[12|10:5]	rs2	rs1	001	imm[4:1|11]	1100011	BNE
imm[12|10:5]	rs2	rs1	100	imm[4:1|11]	1100011	BLT
imm[12|10:5]	rs2	rs1	101	imm[4:1|11]	1100011	BGE
imm[12|10:5]	rs2	rs1	110	imm[4:1|11]	1100011	BLTU
imm[12|10:5]	rs2	rs1	111	imm[4:1|11]	1100011	BGEU
imm[20|10:1|11|19:12]				rd	1101111	JAL
imm[11:0]		rs1	000	rd	1100111	JALR

imm[11:0]		rs1	010	rd	0000011	LW
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	SW

2. RISC-V 处理器设计

2.1 单周期处理器顶层设计

CPU 由控制器和数据通路构成，其中数据通路是指处理器中数据流通的路径。不同类型的指令的数据通路不一样，具体由控制器产生的控制信号决定。

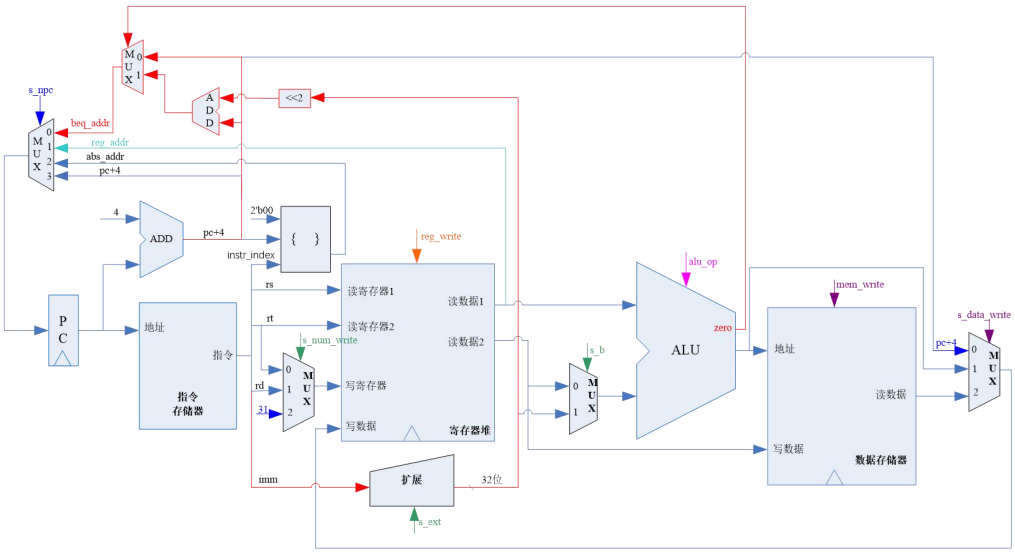

图 2.1 单周期处理器图

如上图所示，CPU 执行指令的过程如下：

（1）pc 做为地址，输入到指令存储器，读取一条指令出来。这条指令是一个 32bit 的二进制数，里面包含了将要执行的指令的所有信息，包括指令类型（opcode），指令执行的功能（func3,func7）,立即数部分（imme）,源寄存器（rs1,rs2），目的寄存器（rd）。

（2）将读出来的指令进行译码，输入到不同的部件。比如，rs1,rs2,rd 输入到寄存器堆，将立即数部分进行立即数扩展，将 opcode 输入到控制部件，产生控制信号。将 func3,func7 输入到子控制模块，产生 ALU 的控制信号。

（3）译码出来的寄存器号，输入到寄存器堆，读出寄存器号对应的数据 Read_data1 和 Read_data2。

对于 R-type 指令，进行的操作就是两个寄存器的数据进行运算。

对于 I-type 指令，进行的操作是一个寄存器的数据与立即数进行运算。

因此这里要产生立即数，并且需要一个二选一选择器进行数据的选择。

（4）ALU 即是运算单元，对输入的两个数据进行运算操作并且将运算结果输出，具体进行什么运算，由 ALU_control 产生的控制信号决定。

（5）将计算结果写回寄存器，写入到目标寄存器号对应的寄存器中。

（6）对于访存指令，如 lb,lh,lw,sb,sh,sw,指令，还需要与数据存储器进行数据交换。

对于 load 指令，执行的操作是将数据从数据存储器的某个地址读出数据并讲这个数据写回到寄存器。

对于 store 指令，执行的操作是将寄存器的数据写入到数据存储器的某个地址内。

这个数据存储器的地址以基地址加偏移的形式给出，基地址从寄存器中取得，偏移量从立即数中得到，因此 ALU 进行加法运算的结果便是数据存储器的地址。

因此 ALU 的运算结果有两个去向，一是写回寄存器，二是作为数据存储器的地址。

（7）对于 load 指令，是将数据存储器的数据写回到寄存器。

因此寄存器的数据来源有两个，一是 ALU 的运算结果，二是从数据存储器读出的数据。

（8）对于不进行跳转的指令，每个周期 pc+4,指令按顺序执行，因此需要一个加法器。一条指令有 32bit,CPU 的内存空间按字节编址，所以一条指令 4 个字节实际上占了 4 个地址空间。

（9）对于条件跳转指令，如 BEQ,BNE 等，满足跳转的条件时，跳转的新地址由 pc 加左移一位的立即数产生，又需要一个加法器。因此，pc 的来源有两个，一是顺序执行，pc+4,二是条件跳转，pc+(imme2)。

（10）对于无条件跳转指令，如 jal,pc+4 写回寄存器，将 pc+(imme2)作为新地址。对于 jalr,将 pc+4 写回寄存器，将源寄存器的数据与立即数相加并将最低位置 0，作为下一个 pc。

因此，写入寄存器的数据来源变成 3 个：ALU 的运算结果，数据存储器读出的数据，pc+4。

pc 的来源也变成 3 个：pc+4,pc+imme,Read_data1+imme。

（11）再考虑 U-type，即 lui,auipc。对于 lui,将立即数进行处理，写入寄存器。对于 auipc,将 pc+imme 写入寄存器。

因此，写入寄存器的数据来源变成 5 个，分别是：ALU 的运算结果，数据存储器读出的数据，pc+4,lui 的立即数 ，auipc 的 pc+imme。

2.1.1 指令存储器

指令存储器属于只读存储器（ROM）,可读不可写，通常有两种实现方式：用硬件描述性语言设计或是用 FPGA 工具调用专用的 ROM IP 核。而第一种设计方式可以使代码的通用性较高，所以这部分采用硬件描述性语言进行设计。

2.1.2 数据存储器

数据存储器属于随机读写存储器（ram）,可读可写，同样是两种实现方式。这里用硬件描述语言设计。

由于指令集中包含多位宽的读写指令，所以这里数据存储器的设计也应该支持多位宽读写。

2.1.3 寄存器堆

寄存器堆由快速的静态随机读写存储器（sram）实现，这种 ram 可以多路并发访问不同的寄存器。寄存器堆由 32 个寄存器阵列组成，其中 0 号寄存器的值恒为 0，只读不写。

2.1.4 译码模块

该模块对指令存储器输出的 32 位指令进行译码，得到 opcode,Rs1,Rs2,Rd, Func3,func7 等信息

2.1.5 ALU 模块

ALU 模块主要进行数据的运算，根据 ALU 的控制模块产生的控制信

ALU_CTL 决定 ALU 进行的运算类型。

2.1.6 PC 寄存器

PC 寄存器用以更新 pc 的值。顺序执行时，pc_new=pc+4,条件跳转时，pc_new=pc+imme。jalr 时，pc_new=data(Rs1)+imme。

2.2 单周期处理控制器信号产生

主控制器：

由于不同类型的指令所经过的数据通路不同，所以需要有控制信号控制数据通路，使得数据经过正确的通路，得到正确的运算结果。本设计将控制器分为两级控制，主控制器产生大部分的控制信号，子控制器是 ALU 控制器，产生控制 ALU 进行正确运算的信号。

子控制器

子控制器根据主控制器产生的 ALUop 信号，结合 func3 和 func7 信号来产生 ALUctl 信号。

将主控制模块和子控制模块进行实例化，得到控制模块。

2.3 信号在五级流水线中的传递

在此流水线中一条指令的传递分为以下步骤：

取值：取指令是指将指令从存储器中都出来的过程。

译码：指令译码是指将从存储器中取出的指令进行翻译的过程。经过译码之后得到的指令需要的操作数寄存器索引，可以使用此索引从通用寄存器组（Register File，Regfile）中将操作数读出。

执行：指令译码之后所需要进行的计算类型都已得知，并且已经从通用寄存器组中读取出了所需的操作数，那么之后便进行指令执行。指令执行是指对指令进行真正运算的过程。在“执行”阶段的最常见部件为算术逻辑部件运算器（ALU），作为实施具体运算的硬件功能单元。

访存：存储器访问指令往往是指令集中最重要的指令类型之一，访存是指存储器访问指令将数据从存储器中读出，或写入存储器的过程。

写回：写回是指将指令执行的结果写回通用寄存器组的过程。如果是普通运算指令，该结果值来自于“执行”阶段计算的结果；如果是存储器读指令，该结果来自于“访存”阶段从存储器中读取出来的数据。

在处理器中采用流水线设计有助于提高处理器的性能。以上述五级流水线为例，由于前一条指令在完成了“取指”进入“译码”阶段后，下一条指令马上就可以进入“取指”阶段，以此类推。如果流水线没有停顿，理论上可以取得每个时钟周期都完成一条指令的性能。

本部分设计的 cpu 能够支持 20 多条指令，具体主要从 ALU 操作、控制信号和指令的关系、CPU 各部分的硬件设计等方面来对流水线处理器的设计进行说明，最后给出对整个流水线处理器进行了仿真验证。

2.4 五级流水线 RISC-V 处理器设计

一般来说，一个 CPU 在处理指令时需要经过以下几个步骤：

取指令(IF)：根据程序计数器 PC 中的指令地址，从存储器中取出一条指令，同时，PC 根据指令字长度自动递增产生下一条指令所需要的指令地址，但遇到“地址转移”指令时，则控制器把“转移地址”送入 PC，当然得到的“地址”需要做些变换才送入 PC。

指令译码(ID)：对取指令操作中得到的指令进行分析并译码，确定这条指令需要完成的操作，从而产生相应的操作控制信号，用于驱动执行状态中的各种操作。

指令执行(EXE)：根据指令译码得到的操作控制信号，具体地执行指令动作，然后转移到结果写回状态。

存储器访问(MEM)：所有需要访问存储器的操作都将在这个步骤中执行，该步骤给出存储器的数据地址，把数据写入到存储器中数据地址所指定的存储单元或者从存储器中得到数据地址单元中的数据。

结果写回(WB)：指令执行的结果或者访问存储器中得到的数据写回相应的目的寄存器中。本次设计的流水线处理器连接框图如图 2.2 所示：

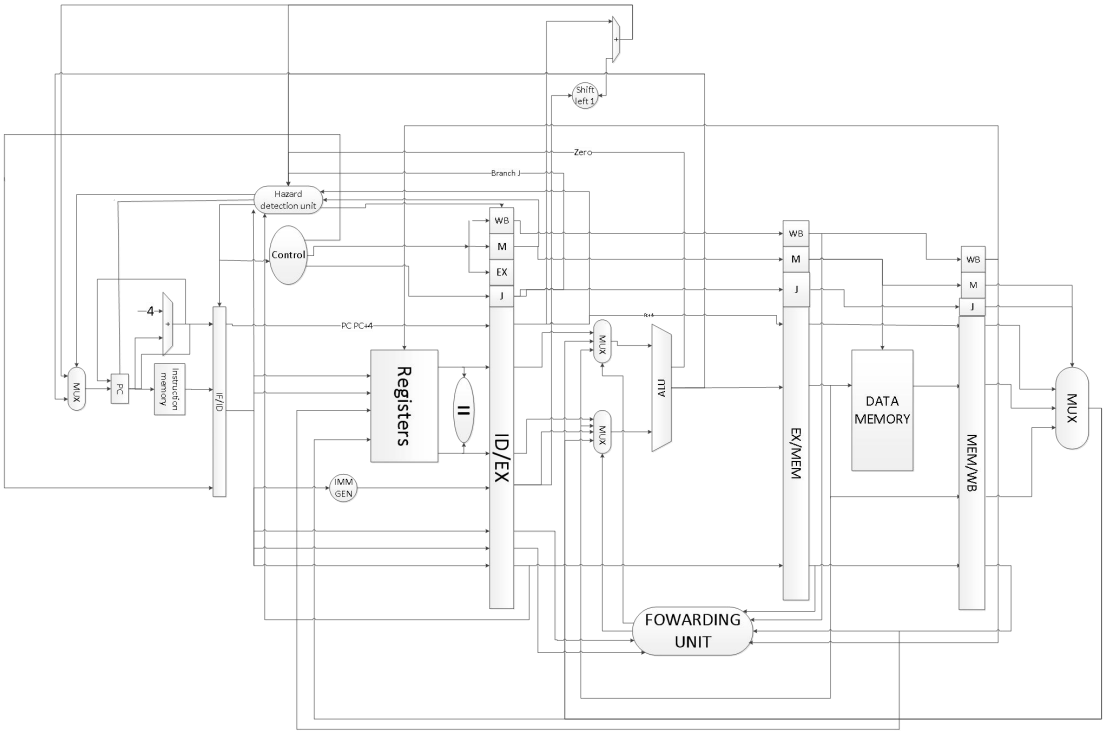

图 2.2 流水线 CPU 数据通路和控制线路图

2.4.1 顶层模块名

CPU_TOP：例化四个模块。包括：CPUlSX 模块、指令存储器模块、数据存储器模块、外设模块。

CPULSX：其中例化了五级流水线中各个部分。

ram_ins：指令存储器模块。

ram_data：数据存储器模块。

外设模块：例化了 2 个模块包括流水灯、数码管。

2.4.2 顶层 I/O 定义

表一 五级流水线 RISC-V 处理器顶层输入/输出信号定义

序号	信号名	输入/输出	位宽	说明	信号源	信号目标
1	clk	input	1	时钟，上升沿有效	TOP 顶层	
2	rst_n	input	1	复位，低电平有效	TOP 顶层	

3	Instru	input	32	时钟上升沿采样指令，根据 PC 值从外部存储获得指令	指令存储器	
4	data_in	input	32	当 data_rd 为高电平，根据 data_addr 得到数据，在时钟上升沿采样	数据存储器	
5	Data_rd	output	1	高电平有效		数据存储器
6	Data_out	output	32	在 Data_wr 有效，将 Data_out 数据写入到 data_addr 指定的数据存储单元中		数据存储器
7	Data_wr	output	1	高电平有效		数据存储器
8	pc_out	output	32	程序计数器，根据 PC 从指令存储器取指令		指令存储器

2.4.3 顶层结构图

1）画出顶层 RTL 结构图

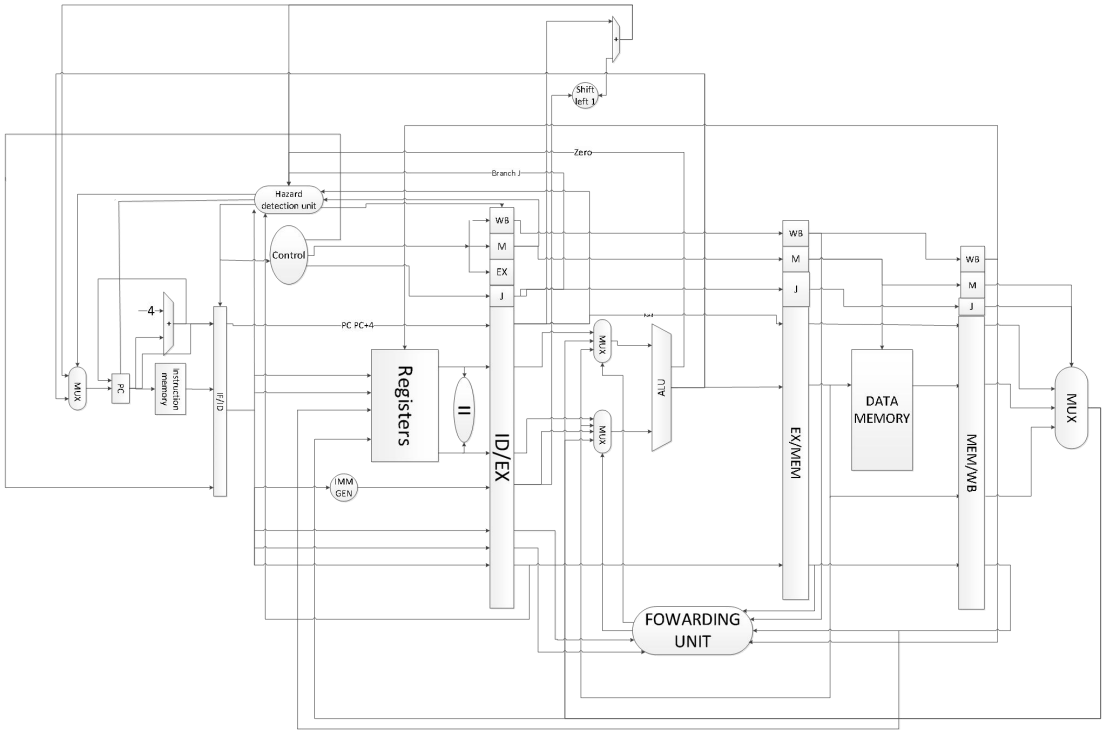

图 2.3 五级流水线 RISC-V 处理器 RTL 结构图

2）顶层 RTL 结构图中各子模块的功能定义

PC：完成计数功能。

Control：读取操作码，输出相应位的控制信号。

ALU：根据 ALU control 的输出值对 source1 和 ource2 进行相应的逻辑运算。

ALU control：根据 ALUOp 和指令源码的 30 位和 14-12 位（instruction[30], instruction[14:12]）输出相应的 ALUCtrl 信号。

指令存储器：用于存储指令，给定地址直接输出指令。

寄存器堆：为 32 个 32 位寄存器。

数据存储器：存储数据。

Forwarding unit：解决流水线运作时可能产生的数据冒险。

Hazard detection unit：解决流水线运作时可能产生的控制冒险。

3. 子模块设计

3.1 PC 寄存器设计

功能：程序计数器

输入输出信号定义：

表一：PC 寄存器存储设计信号说明

参数名	方向	位宽	说明
clk	输入	1	时钟信号
rest	输入	1	复位信号
sel	输入	1	选择信号
source	输入	32	立即数输入
en	输入	1	使能信号
pc	输出	32	输出 pc 值
pcadd4	输出	32	输出 pc+4 的值

设计：当选择信号 sel 为高电平的时，直接将立即数给 pc 输出，当当选择信号 sel 为低电平时 pc+4 再输出；输出信号 pcadd4 是将 pc 的值始终加 4 输出。

3.2 Control 设计

功能：读取操作码，输出相应位的控制信号

输入输出信号定义：

表二：Control 设计信号说明

参数名	方向	位宽	说明
opcode	输入	6	操作码
Branch	输出	1	分支控制 1：表示可能进行分支/0：不分支
MemRead	输出	1	读数据存储器 1：读取数据
MemtoReg	输出	1	1：数据存储器->Reg/ 0：ALU 结果->Reg
ALUOp	输出	2	ALU 操作码

MemWrite	输出	1	写数据存储器 1：写入数据
ALUSrc	输出	1	ALU 第二个操作数来源 1：立即数 0：寄存器堆
jal_select	输出	1	跳转指令时置位
RegWrite	输出	1	写寄存器堆 1：写使能

设计：读取 7 位操作码，对多路选择器、ALU Ctrl、写寄存器堆、读写数据存储器的使能信号。详细控制信号参考控制信号与相对应指令之间的相互关系表。

3.3 ALU 设计

功能：根据 ALU control 的输出值对 source1 和 ource2 进行相应的逻辑运算

输入输出信号定义：

表三：ALU 设计信号说明

参数名	方向	位宽	说明
source1	输入	32	读取出的数据 1
source2	输入	32	读取出的数据 2
ALUCtrl	输入	6	ALUCtrl 输出的控制信号
Zero	输出	1	输出结果为 0 是的标志信号
result	输出	32	经过 ALU 计算后的结果

设计：当 ALUCtrl 为 5'b00100 时，ALU 执行加法。

当 ALUCtrl 为 5'b00101，5'b00111 时，ALU 执行有符号减法。

当 ALUCtrl 为 5'b11101 时，ALU 执行与运算。

当 ALUCtrl 为 5'b11100 时，ALU 执行或运算。

当 ALUCtrl 为 5'b11110 时，ALU 执行异或运算。

当 ALUCtrl 为 5'b10000 时，ALU 执行无符号小于。

当 ALUCtrl 为 5'b10100 时，ALU 执行有符号小于<。

当 ALUCtrl 为 5'b10001 时，ALU 执行无符号大于等于>=。

当 ALUCtrl 为 5'b10101 时，ALU 执行有符号大于等于>=。

当 ALUCtrl 为 5'b01101 时，ALU 执行逻辑右移。

当 ALUCtrl 为 5'b01100 时，ALU 执行逻辑左移。

当 ALUCtrl 为 5'b01110 时，ALU 执行算数右移。

当 ALUCtrl 为 5'b00110 时，ALU 执行 Zero 强制输出 1。

3.4 ALU control 设计

功能：根据 ALUOp 和指令源码的 30 位和 14-12 位（instruction[30], instruction[14:12]）输出相应的 ALUCtrl 信号。

输入输出信号定义：

表四：ALU control 设计信号说明

参数名	方向	位宽	说明
instruction	输入	7	Instruction 的 30 位和 14-12 位
ALUOp	输入	2	ALUCtrl 的控制信号
ALUCtrl	输出	6	ALU 的控制信号

设计：

当 ALUOp 为 010 时进行 R 型指令操作，再根据 funct7 字段和 funct3 字段来决定具体的操作。

当 ALUOp 为 001 时进行 B 型指令操作，再根据 funct3 字段来进行具体操作的区分。

当 ALUOp 为 011 和 100 时进行 I 型指令操作，再根据 funct3 字段来进行具体操作的区分。

当 ALUOp 为 001 时进行 S 型指令操作，再根据 funct3 字段来进行具体操作的区分。

当 ALUOp 为 101 时进行 U 型指令操作，再根据 funct7 字段和 funct3 字段来决定具体的操作。

当 ALUOp 为 110 时进行 U-type 指令操作中的 jal 指令。

3.5 指令存储器设计

功能：用于存储指令，给定地址直接输出指令

输入输出信号定义：

表五:指令存储器设计信号说明

参数名	方向	位宽	说明
address	输入	32	指令存储地址
Instruction	输出	32	数据输出

设计：address 是指令存储地址，instruction 是 32 位的指令输出。

3.6 寄存器堆设计

功能：32 个 32 位寄存器

输入输出信号定义：

表六：寄存器堆设计信号说明

参数名	方向	位宽	说明
clk	输入	1	时钟信号
ReadReg1	输入	5	读取的第一组寄存器
ReadReg2	输入	5	读取的第二组寄存器
WriteReg	输入	5	写入寄存器
Writedata	输入	32	写入数据
RegWrite	输入	1	写使能
ReadData1	输出	32	第一组寄存器读出的数据
ReadData2	输出	32	第二组寄存器读出的数据

设计：给定两个要读取的寄存器（ReadReg1、ReadReg2），在相应的读数据输出端（ReadData1、ReadData2）输出数据。给定要写的数据寄存器（WriteReg）、写入的数据（Writedata），在使能写入后，下一个时钟的上升沿将数据写入该寄存器。

3.7 数据存储器设计

功能：存储数据

输入输出信号定义：

表七：数据存储器设计信号说明

参数名	方向	位宽	说明
clk_data	输入	1	存储数据时钟
address	输入	32	读取的地址
Writedata	输入	32	写入的数据
MemWrite	输入	1	写使能
MemRead	输入	1	读使能
ReadData	输出	32	数据输出端

输入输出信号定义：给定要读取的地址（address）和读使能信号（MemRead），在数据输出端（ReadData1、ReadData）直接输出数据。给定要写的数据地址（address）、写入的数据（Writedata），在使能写（MemWrite）后，下一个时钟的上升沿将数据写入该寄存器。

3.8 前递单元设计

功能：解决流水线运作时可能产生的数据冒险。

输入输出信号定义：

表八：前递单元（Forwarding unit）设计信号说明

参数名	方向	位宽	说明
Reg32Source1_From_EX	输入	5	当前执行单元需要用到的寄存器
Reg32Source2_From_EX	输入	5	当前执行单元需要用到的寄存器
Reg32Source1_From_ID	输入	5	译码单元译码用到的寄存器 1
Reg32Source2_From_ID	输入	5	译码单元译码用到的寄存器 2
WriteReg_From_MEM	输入	1	当前 MEM 级是否要回写寄存器

Writedata_From_MEM	输入	5	当前 MEM 级要回写的寄存器
WriteReg_From_WB	输入	1	当前 WB 级是否要回写寄存器
Writedata_From_WB	输入	5	当前 WB 级要回写的寄存器
ALUSrc_From_ID	输入	1	ALU 的第二操作数来源
ALUSrc1Ctrl_To_ALUSrc1	输出	2	控制 ALU 第一个操作数的来源
ALUSrc2Ctrl_To_ALUSrc2	输出	2	控制 ALU 第二个操作数的来源
IDEXRegSrc1Ctrl_To_IDEXRegSrc1MUX	输出	2	解决回写与译码冲突问题（详情见操作过程 4）
IDEXRegSrc2Ctrl_To_IDEXRegSrc2MUX	输出	2	解决回写与译码冲突问题（详情见操作过程 4）
EXMEMRegSrc2Ctrl_To_EXMEMRegSrc2MUX	输出	2	解决 SW 指令问题（详情见操作过程 5）

前递单元（Forwarding unit）操作数：

1. Reg32 Source1(5 位)由执行单元提供。
2. Reg32 Source2(5 位)由执行单元提供。
3. （是否写入寄存器（1bit）和要写入寄存器的位置（5bit））由访存单元提供
4. （是否写入寄存器（1bit）和要写入寄存器的位置（5bit））由回写单元提供
5. ALUSrc（ALUSrc2 来自谁？来自立即数还是寄存器）

前递单元（Forwarding unit）操作结果：

1. 控制 ALU Source1 数据来源（2bit）
2. 控制 ALU Source2 数据来源（2bit）

前递单元（Forwarding unit）操作过程：

1. 比较（Reg32 Source1(5 位)由执行单元提供）与（是否写入寄存器（1bit）和要写入寄存器的位置（5bit））由访存单元提供和（是否写入寄存器（1bit）和要写入寄存器的位置（5bit））由回写单元提供如果有要写回寄存器的值在流水线的（访存）和（回写）单元，则直接控制多路选择器选择该位置的数据作为 ALU Source1。否则多路选择器由 Control 产生的结果控制。

2. ALUSrc==1 时（ALU 第二个操作数来自立即数）不执行 3，否则执行 3。

3.比较（Reg32 Source2(5 位)由执行单元提供）与（是否写入寄存器（1bit）和要写入寄存器的位置（5bit））由访存单元提供和（是否写入寄存器（1bit）和要写入寄存器的位置（5bit））由回写单元提供如果有要写回寄存器的值在流水线的（访存）和（回写）单元，则直接控制多路选择器选择该位置的数据作为 ALU Source2。否则多路选择器由 Control 产生的结果控制。

4. 解决回写与译码冲突问题：

问题说明：在一个数据将要被回写，而当前译码指令用到该寄存器时出现冲突，时钟到来前，要写入的数据在寄存器在数据线上而此时译码单元却将当前寄存器的值读出。下个时钟到来后错误的值被给到执行单元。解决方案有两种：

- 加入两个多路选择器，在译码时判断是否存在数据冲突，当发生时将当前正在回写数据线上的数据送给 IDEX 寄存器。
- 取消寄存器堆的时钟信号，但会引入大量锁存器。

我们采取第一种方法解决。

5. 解决写数据存储器与 ALU 结果冲突问题：

问题说明：当执行如下两条指令时

Addi x1,x0,5

Sw x1,x2,0

在 ALU 执行第一条指令时 Sw 将正在译码阶段，此时 x1 数据在 ALU 的输出端，下个周期 x1 的值进入执行单元（x1 为上个周期的 x1），此时如果不引入旁路会导致上一时刻的 x1 继续向下传递，导致写入存储器的数据错误。因此我们在执行阶段引入旁路选择，保证向存储器中写入正确的数据。

3.9 冒险检测设计

功能：解决流水线运作时可能产生的控制冒险。

输入输出信号定义：

表九：冒险检测单元（Hazard detection unit）设计信号说明

	参数名	方向	位宽	说明
分支预测部分端口	PC+4	输入	32	下一个指令的 pc 值
	ALUresult	输入	32	Alu 计算的结果
	PC+imm	输入	32	Pc 加立即数
	Branch	输入	1	Aluctrl 的分支信号
	j	输入	1	用来区分 j 指令
	Zero	输入	1	分支跳转标志信号
	Pc_source_mux	输出	1	PC 跳转地址选择
	Jump_control	输出	1	跳转控制
空指令部分	Sr1	输入	5	源寄存器地址 1
	Sr2	输入	5	源寄存器地址 2
	rd	输入	5	写回的目标寄存器
	MemRead	输入	1	读使能
	Pc_en	输出	1	Pc 使能信号
	If/id_en	输出	1	if/id 寄存器使能信号

冒险检测单元（Hazard detection unit）操作数:

1. (IF|ID)寄存器传出的要读取的寄存器（两个，5bit*2）
2. (ID|EX)传出的要回写的寄存器（1 个，5bit）
3. (ID|EX)传出的是否读 Mem 信号（1 个，1bit）
4. Zero&&Branch
5. J
6. PC+imm)
7. EX 阶段的 PC+4
8. ALUResult

冒险检测单元（Hazard detection unit）操作结果:

1. PC 使能信号。
2. 清空 IF|ID、ID|EX、EX|MEM 寄存器值（同步清空）

3.强制 PC 加载目标在地址

冒险检测单元（Hazard detection unit）操作过程:

空指令：在冒险检测单元 发现当前译码单元操作的两个寄存器中用到执行单元要回写的寄存器的值时，控制流水线 PC 保持当前值，(IF|ID)，(ID|EX)保持当前值。

清空流水线操作：定义当发生控制冲突时需清空流水线操作，强制清除流水线寄存器值（清空 IF|ID、ID|EX）。因此，流水线寄存器必须要有复位，可能并不是所有的复位值都为 0。

表十：分支预测部分控制信号表格信号说明

	B 指令	JAL 指令	JALR 指令
Branch	1	0	1
J	0	1	1

4 RISC-V 处理器仿真验证

4.1 验证方案描述

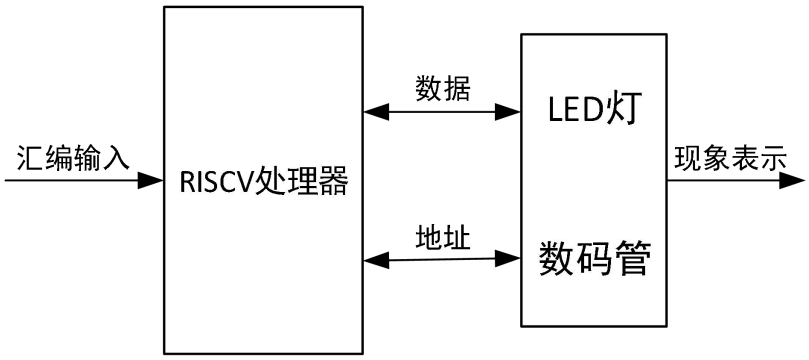

图 4.1 验证方案描述图

4.2 测试用例

4.2.1 单条指令测试

（1）通过计算 10 和 160 相加对 addi、slli、add、sw、jal 指令进行测试。

addi x1,x0,0x01

slli x1,x1,31 （在 x1 内放入地址 8000_0000）

addi x2,x0,0x0a （addi 指令将加数 0x0a 放入 x2）

addi x3,x0,0xa0 （addi 指令将被加数 0xa0 放入 x3）

loop:

add x4,x2,x3 （add 指令进行加法运算，并将结果放入 x4）

sw x4,0(x1) （将 x4 内计算结果放入指定地址）

jal x0,loop （反复执行有利于抓取结果）

上面指令测试的是一个十六进制 0x0a 和 0xa0 的和，结果转换为二进制为 10101010，转换为 10 进制为 170。下图 4.2 表示的是用 led 灯显示运算结果，因为本实验使用的 FPGA 开发板是双色 led 灯显示，意思也即是说，虽然看上去是四个 led 灯，但实际表示的是 8 个 led 灯。led0 与 led1 共用一个灯表示，只是

led0 显示的是蓝色，而 led1 显示的是红色。以此类推，led2 与 led3 共用一个灯表示，led2 显示的是蓝色，而 led3 显示的是红色。Led4 与 led5 共用一个灯表示，led4 显示的是蓝色，而 led5 显示的是红色。Led6 与 led7 共用一个灯表示，led6 显示的是蓝色，而 led7 显示的是红色。本次结果为 10101010，也就是 led 灯第 1、3、5、7 位亮，也就是下图 4.2 的现象，符合汇编程序运算结果，证明 cpu 指令测试没有问题。下图 4.3 是在数码管上显示运算结果，与运算结果 170 一致。图 4.4 是抓取了相对应的 ILA 数据波形图，与数码管现象完全一致。

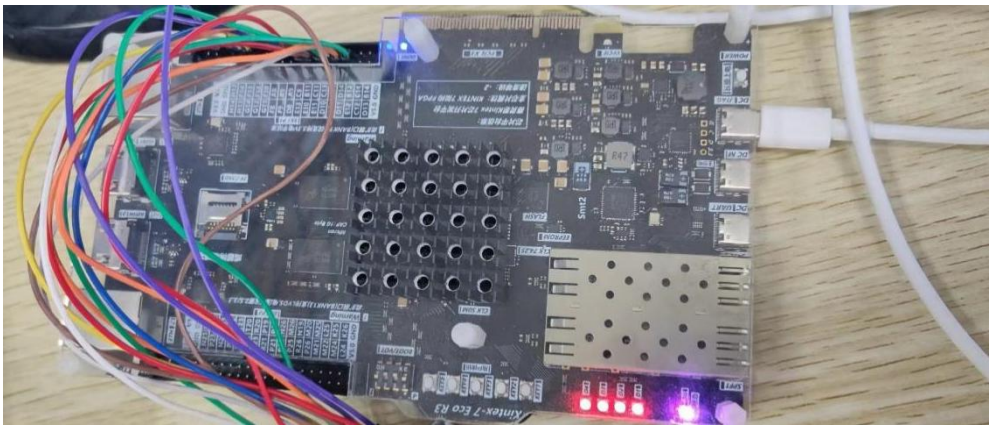

图 4.2 led 灯显示结果图

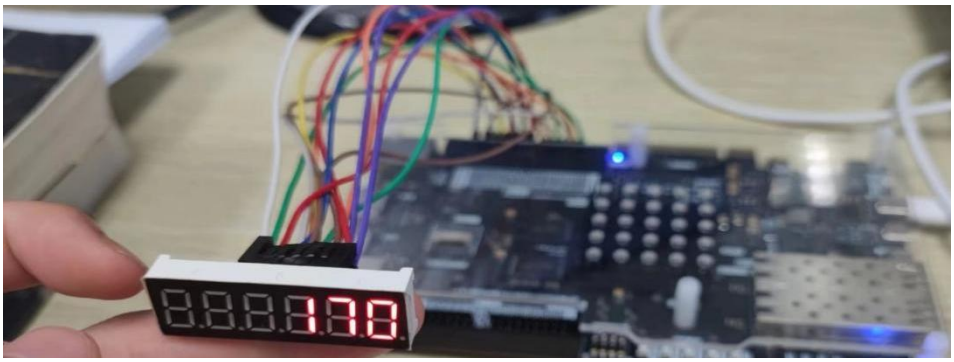

图 4.3 数码管显示结果图

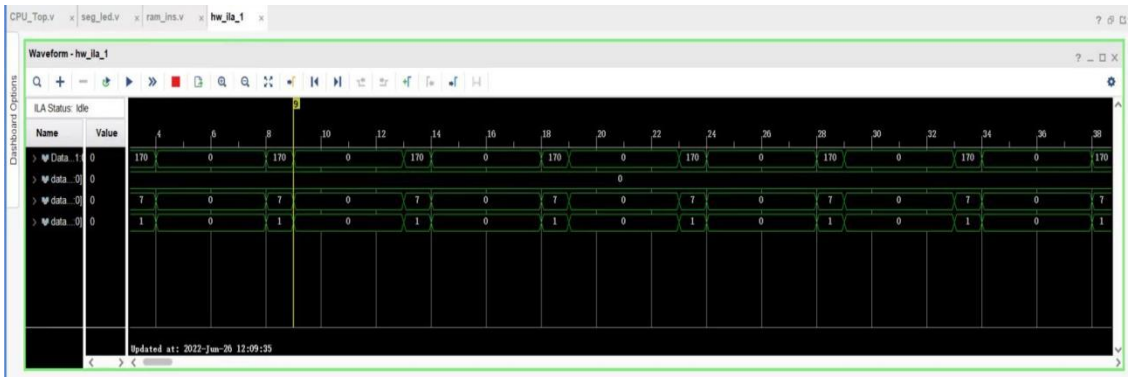

图 4.4 ILA 抓取的数据波形图

（2）测 SRL，结果 0101=5，led 低俩位为蓝。

```
addi x1,x0,0x01
slli x1,x1,31
addi x2,x0,0x0a
addi x3,x0,0x01
loop:
srl x4,x2,x3
sw x4,0(x1)
jal x0,loop
```

下图 4.5 表示的是用 led 灯显示运算结果。

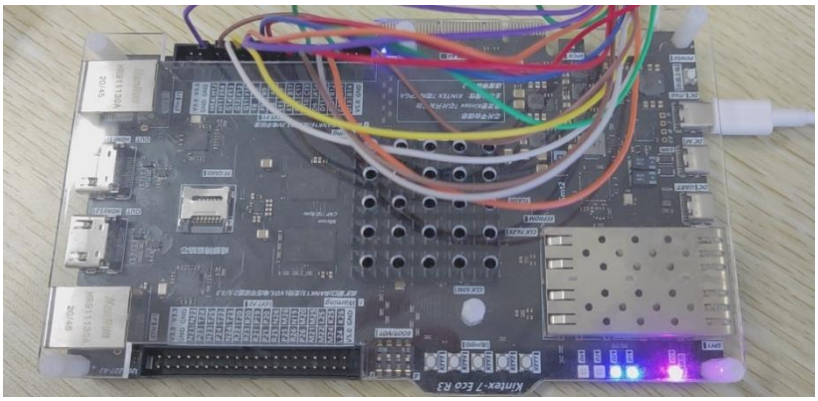

图 4.5 led 灯显示结果图

（3）sll，结果 00010100，亮第 2、3 个蓝灯

```
addi x1,x0,0x01
slli x1,x1,31
addi x2,x0,0x0a
addi x3,x0,0x01
loop:
sll x4,x2,x3
sw x4,0(x1)
jal x0,loop
```

下图 4.6 表示的是用 led 灯显示运算结果。

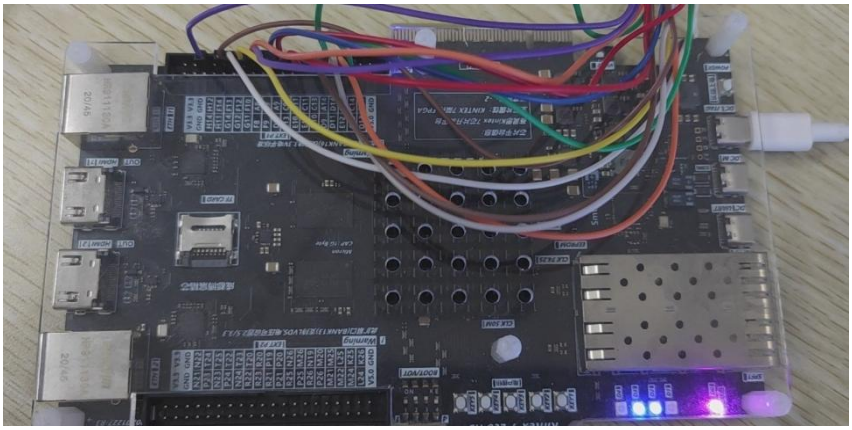

图 4.6 led 灯显示结果图

（4）Sub，结果 00001000=8，亮第二个红灯。

```
addi x1,x0,0x01
slli x1,x1,31
addi x2,x0,0x0a
addi x3,x0,0x02
loop:
sub x4,x2,x3
sw x4,0(x1)
jal x0,loop
```

下图 4.7 表示的是用 led 灯显示运算结果。

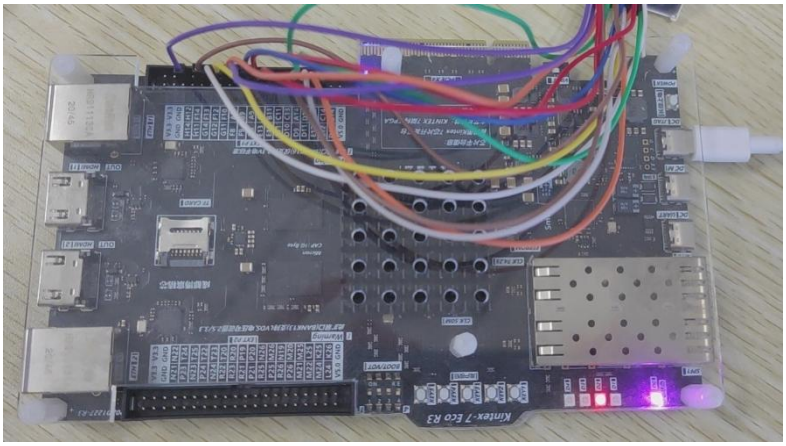

图 4.7 led 灯显示结果图

（5）xor，结果 00001111，亮低两位紫。

addi x1,x0,0x01

slli x1,x1,31

addi x2,x0,0x0a

loop:

xori x4,x2,5

sw x4,0(x1)

jal x0,loop

下图 4.8 表示的是用 led 灯显示运算结果。

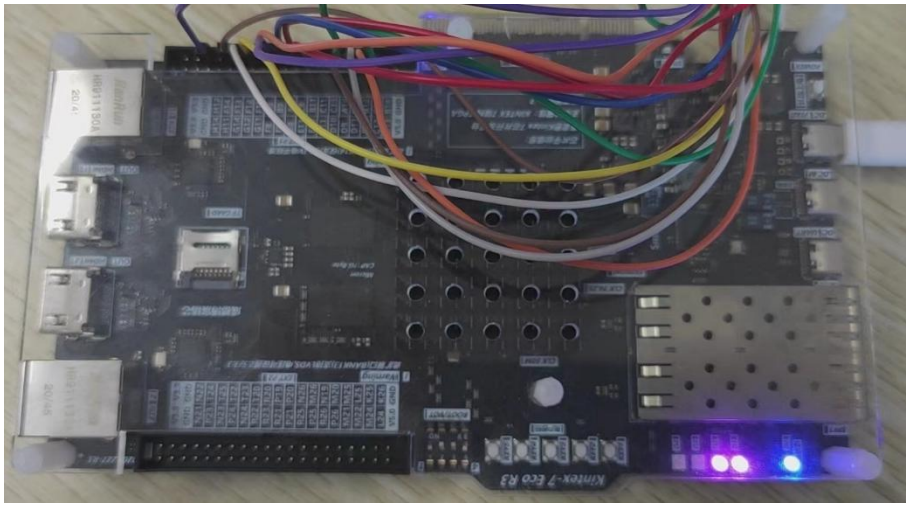

图 4.8 led 灯显示结果图

（6）xori，结果 00001111，亮低两位紫。

addi x1,x0,0x01

slli x1,x1,31

addi x2,x0,0x0a

loop:

xori x4,x2,5

sw x4,0(x1)

jal x0,loop

下图 4.9 表示的是用 led 灯显示运算结果。

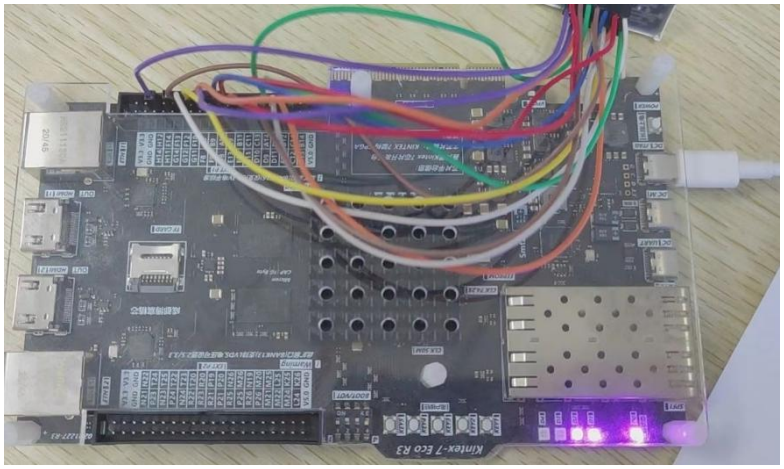

图 4.9 led 灯显示结果图

（7）or，结果 00001110，亮一红一紫。

```
addi x1,x0,0x01
slli x1,x1,31
addi x2,x0,0x0a
addi x3,x0,0x0c
loop:
or x4,x2,x3
sw x4,0(x1)
jal x0,loop
```

下图 4.10 表示的是用 led 灯显示运算结果。

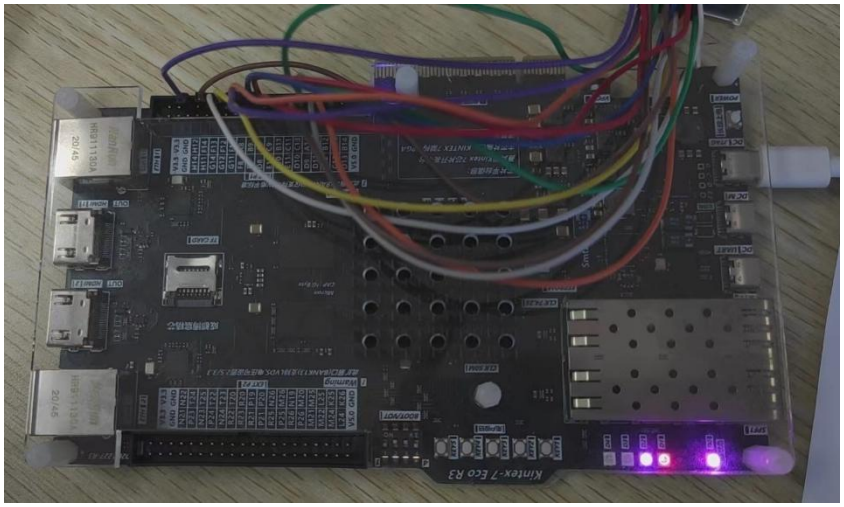

图 4.10 led 灯显示结果图

（8）ori,结果 00001110，亮一红一紫。

```
addi x1,x0,0x01
slli x1,x1,31
addi x2,x0,0x0a
loop:
ori x4,x2,10
sw x4,0(x1)
jal x0,loop
```

下图 4.11 表示的是用 led 灯显示运算结果。

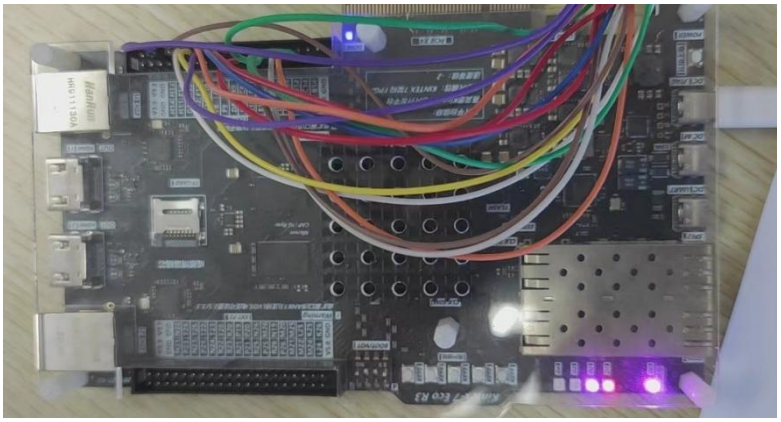

图 4.11 led 灯显示结果图

（9）andi,结果 00001000，亮第二个红。

```
addi x1,x0,0x01
slli x1,x1,31
addi x2,x0,0x0a
loop:
andi x4,x2,10
sw x4,0(x1)
jal x0,loop
```

下图 4.12 表示的是用 led 灯显示运算结果。

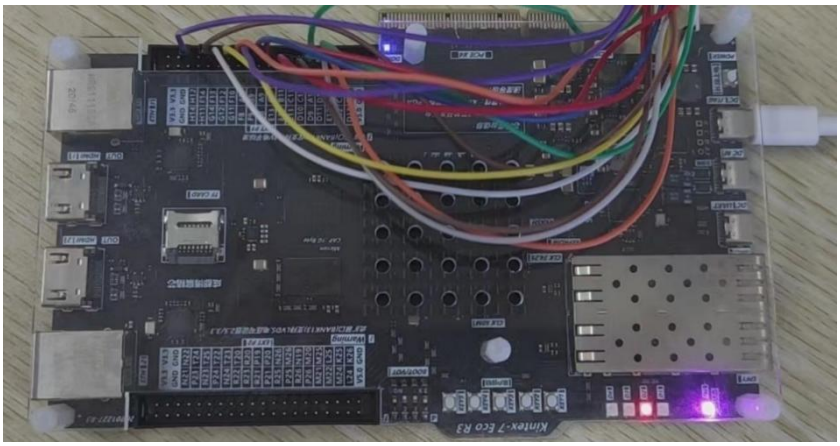

图 4.12 led 灯显示结果图

（10）slti,结果为 1，亮第一个蓝。

addi x1,x0,0x01

slli x1,x1,31

addi x2,x0,0x0a

loop:

slti x4,x2,15

sw x4,0(x1)

jal x0,loop

下图 4.13 表示的是用 led 灯显示运算结果。

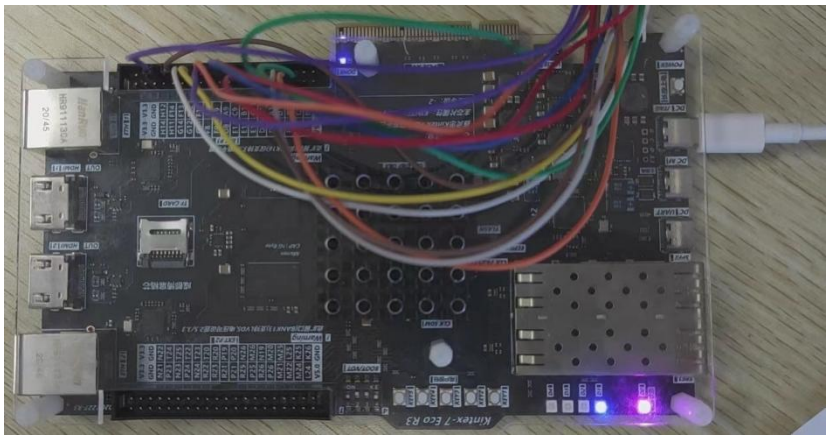

图 4.13 led 灯显示结果图

（11）slt,结果为 1，亮第一个蓝。

addi x1,x0,0x01

slli x1,x1,31

addi x2,x0,0x0a

addi x3,x0,0x0c

loop:

slt x4,x2,x3

sw x4,0(x1)

jal x0,loop

下图 4.14 表示的是用 led 灯显示运算结果。

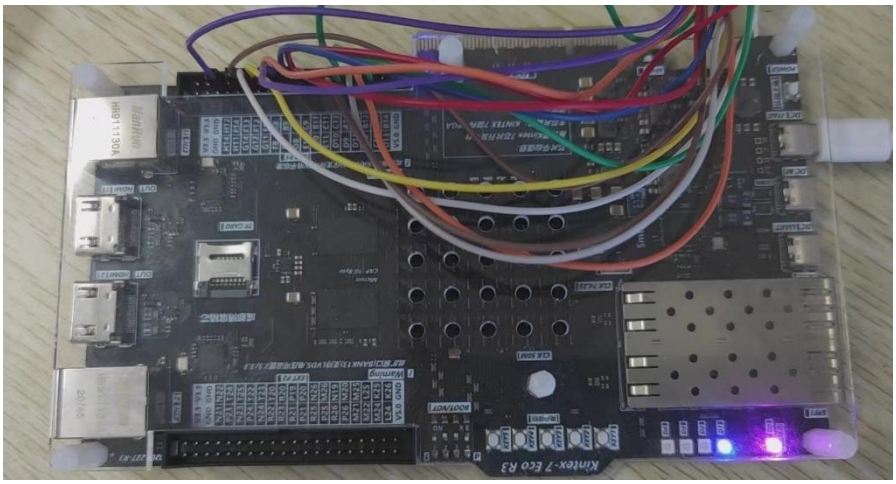

图 4.14 led 灯显示结果图

4.2.2 应用程序测试

递归求累加和：

addi x8,x0,0x01

slli x8,x8,31 （在 x8 内放入地址 8000_0000）

addi x1,x0,200

addi x5,x0,5

addi sp,x0,200

main:

addi x5,x0,5 （求 1+2+3+4+5=15）

addi x6,x0,1

addi x7,x0,0

```
jal x1,fact
addi x1,x0,250
sw x7,0(x8)
jal x0,main   （反复执行有利于抓取结果）
fact:    （将计算所需数据压栈）
 blt x5,x6,return
 sw x1,0(sp)
 addi sp,sp,-4
 sw x5,0(sp)
 addi sp,sp,-4
 addi x5,x5,-1
 jal x1,fact
return: （将数据弹栈并进行累加计算）
 addi sp,sp,4
 lw x5,0(sp)
 addi sp,sp,4
 lw x1,0(sp)
 add x7,x7,x5
jalr x0,x1,0
```

上面汇编程序实现的是递归累加从 1 加到 5。下图 4.15 表示的是将递归累加结果显示在数码管上，结果为 15，和汇编程序应得的结果一致。图 4.16 表示的是通过 Vivado 软件 ILA 抓取的数据波形图，和数码管结果现象一致。

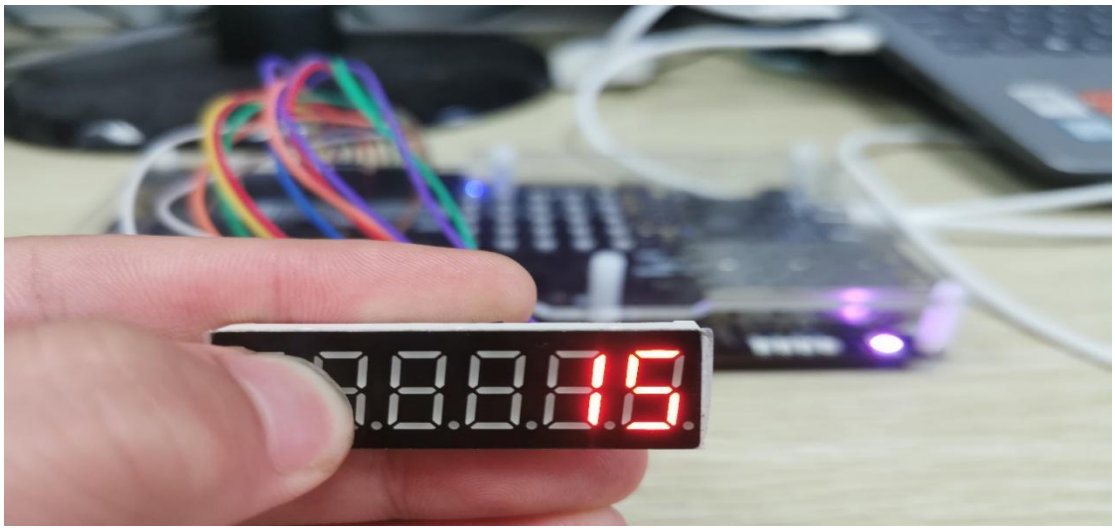

图 4.15 数码管显示结果图

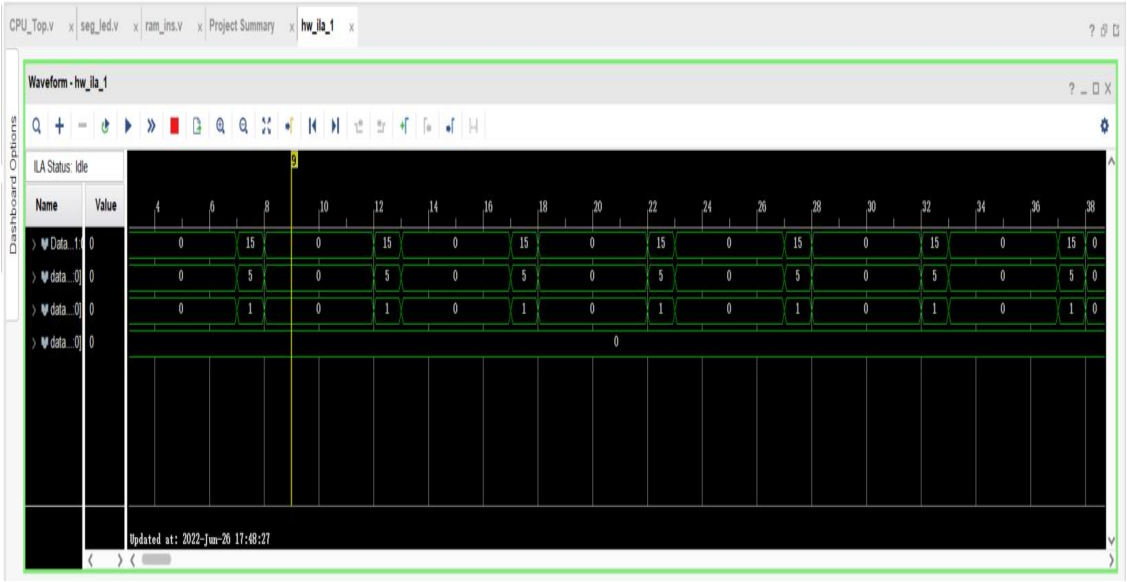

图 4.16 ILA 抓取的数据波形图

5. 项目管理

5.1 开发环境

（1）使用的硬件描述语言

Verilog

（2）仿真工具版本

本实验通过 Vivado 软件 ILA 来进行波形分析

（3）FPGA 开发工具版本

Vivado2019.1

（4）FPGA 开发板型号

赛灵思 Kintex7 开发板

5.2 时间安排

序号	任务	计划完成时间
1	指令集设计	4.10-4.15
2	单周期处理器设计	4.15-5.15
3	流水线处理器设计	5.15-6.15
4	处理器外设设计	6.15-7.5

6. 课程总结

项目完成情况总结
项目总体完成情况：完成预定计划，完成完整的基础处理器设计并对其部分指令进行测试，总体功能也用递归累加进行测试，测试结果均符合预期 存在的问题、下一步工作展望：指令集有待扩充，如 mul 乘法指令等可以使处理器功能更加完善；处理器中可以再加 cache 模块、分支预测模块等来进

一步提升处理器的性能。 收获的经验教训：前期计划一定要制定明确，在设计过程中一定要尽量按照计划的步骤和时间完成相应的任务；理论知识一定要学习的非常清晰才能下手去做，比如流水线的数据通路和前递单元的原理，在对原理还摸棱两可的时候写代码是很容易出错的，很难达到预期效果；测试时也要考虑很多因素，尽量排除其他因素对本次测试目标的影响，如对单条指令测试时应确保辅助时用到的其他指令都没有问题等等。
项目完成情况自评打分

参考文献

[1]武延军,李玲,邢明杰.RISC-V 技术及生态专题前言[J].计算机系统应用,2021,30(11):1-2.

[2]刘先强.基于 RISC-V 的五级流水线处理器的设计与研究[D].山东大学,2021.

[3]蔡斌,徐培欣,张立志,丁立.一种基于 RISC-V 架构的处理器[P].江苏省：CN113687862A,2021.11.23.

[4]潘越,史府鑫,李稚.基于 RISC-V 的卫星通信终端处理器指令集优化研究[C],第十六届卫星通信学术年会论文集,2020:242-250.

[5]崔林海,张子迎,姜占鹏,郭鑫.计算机组成原理与结构[M].哈尔滨工业大学出版社, 2015.01.